Compiler Design

500+ One-Liners for Rapid Revision

IBPS SO (IT Officer) Examination - Complete Preparation Series

100% Syllabus Coverage - MCQ-Oriented - Zero Filler

Compiled by Asabhya Maanav

Table of Contents

1. Compiler Overview - One-Liners

Compiler vs Interpreter vs Assembler

- A compiler translates the **entire** program at once, whereas an interpreter translates **statement by statement**.
- There are **3** core language translators: **compiler**, **interpreter**, and **assembler**.
- An assembler converts **assembly language** into **machine code** with a near **one-to-one** mapping.
- The translator that reports **all errors together** is the **compiler**; the one that stops at the **first** error is the **interpreter**.
- Compiled code executes **faster** while interpreted code is **easier to debug**.
- **Java** uses **both** models: a compiler to **bytecode** and the **JVM** interpreter/JIT at runtime.
- A **preprocessor** runs **before** the compiler to handle **macros** and **file inclusion** like `#include` and `#define`.
- A **linker** combines **object modules** and resolves **external references**.
- A **loader** places the **executable** into **memory** for execution.
- The 2 disadvantages of an interpreter are **slow execution** and **re-translation** every run.

Phases of a Compiler

- A compiler has **6** phases: **lexical**, **syntax**, **semantic**, **intermediate code**, **optimization**, and **code generation**.
- The **first** phase of a compiler is **lexical analysis** and the **last** is **code generation**.
- The **2** non-phase components spanning all phases are the **symbol table** and the **error handler**.
- The phase producing **tokens** is **lexical analysis**.
- The phase producing a **parse tree** is **syntax analysis**.
- The phase producing an **annotated/decorated tree** is **semantic analysis**.
- The phase producing an **IR** (three-address code) is **intermediate code generation**.
- The phase improving code without changing meaning is **code optimization**.
- The phase emitting **target machine code** is **code generation**.
- A **lexical** error is an **invalid token** like `@` or `1abc`.
- A **syntax** error is a **grammar violation** like a **missing semicolon**.
- A **semantic** error is a **type mismatch** or use of an **undeclared variable**.
- The mnemonic for phase order is **"La Sa Se In Op Co"**.

Front End vs Back End

- The **front end** (analysis) has **4** parts: **lexical**, **syntax**, **semantic**, and **intermediate code generation**.
- The **back end** (synthesis) has **2** parts: **code optimization** (machine-dependent) and **code generation**.
- The front end depends on the **source language**; the back end depends on the **target machine**.
- Splitting front/back ends turns **m*n** compiler effort into **m+n** via a shared **IR**.
- The **intermediate code generation** phase belongs to the **front end**, not the back end.

Symbol Table, Errors, Passes, Bootstrapping

- A symbol table stores **name**, **type**, **scope**, **size**, and **memory location** of identifiers.
- The fastest symbol-table implementation has **O(1)** average lookup using a **hash table**.
- There are **4** error-recovery strategies: **panic mode**, **phrase-level**, **error productions**, and **global correction**.
- **Panic-mode** recovery skips tokens until a **synchronizing token** like `;` appears.
- A **single-pass** compiler scans **once**; a **multi-pass** compiler scans **multiple** times for better optimization.
- The number of **phases** is **6** regardless of the number of **passes**.

- **Bootstrapping** writes a compiler for a language **in that same language**.
- A **cross-compiler** has **host != target** machine.

2. Lexical Analysis - One-Liners

Scanner Role and Tokens

- The **lexical analyzer** (scanner) converts the **character stream** into a **token stream**.
- The scanner removes **whitespace** and **comments** and tracks **line numbers**.
- The **3** key terms are **token** (class), **pattern** (rule), and **lexeme** (actual text).
- A token is emitted as **<token-name, attribute-value>**.
- Many **lexemes** can map to one **token**, e.g., `x` and `count` are both **id**.
- For `int count = 10;` there are **5** tokens: **int**, **count**, **=**, **10**, and **;**.
- The scanner uses the **longest match (maximal munch)** rule with **keyword priority** on ties.

Regular Expressions and Finite Automata

- Tokens are specified by **regular expressions** and recognized by **finite automata**.
- Regex operator precedence (high to low) is **star ***, then **concatenation**, then **union |**.
- `r+` means **one or more**; `r*` means **zero or more**; `r?` means **zero or one**.
- An identifier pattern is **letter(letter|digit)***.
- Regular expressions **cannot** match **balanced parentheses**; that needs a **CFG**.
- The regex-to-scanner pipeline is **Regex -> NFA -> DFA -> minimized DFA**.
- **Thompson's construction** converts **regex to NFA**; **subset construction** converts **NFA to DFA**.
- A DFA built from an n-state NFA can have up to **2^n** states.
- **NFA** and **DFA** accept the **same** class of languages (**regular**).
- The scanner is implemented as a **DFA** for **speed**.

Buffering, LEX, Symbol Table

- Input buffering classically uses **2** buffers with a **sentinel** (`eof`) and **2** pointers (**lexemeBegin**, **forward**).
- The **sentinel** avoids an **end-of-buffer** test on every character.
- **LEX/Flex** is a **lexical-analyzer generator** producing the function **yylex()**.
- A LEX program has **3** sections separated by %%: **declarations**, **rules**, **auxiliary functions**.
- On equal-length matches, LEX picks the **rule listed first**.
- The scanner enters **identifiers** into the **symbol table** and returns a **pointer** as the attribute.
- **Keywords** are usually **pre-loaded** into the symbol table to distinguish them from identifiers.

3. Grammars for Parsing - One-Liners

Context-Free Grammars

- A CFG is a **4-tuple (V, T, P, S)**: **non-terminals**, **terminals**, **productions**, **start symbol**.
- A CFG production has **exactly one** non-terminal on its **LHS**.
- Programming-language **syntax** uses **CFG (Type-2)**; **tokens** use **regular grammar (Type-3)**.
- CFGs can express **nested/recursive** structures that **regex** cannot.

Derivations and Parse Trees

- A **leftmost** derivation expands the **leftmost** non-terminal first; **rightmost** expands the **rightmost**.
- **Top-down** parsing simulates a **leftmost** derivation.
- **Bottom-up** parsing simulates a **rightmost derivation in reverse**.
- A parse tree's **root** is the **start symbol** and its **leaves** form the **yield**.
- Each parse tree has a **unique** leftmost and a **unique** rightmost derivation.

Ambiguity, Left Recursion, Left Factoring

- A grammar is **ambiguous** if a string has **two or more** parse trees.
- General grammar ambiguity is **undecidable**.
- The **dangling-else** problem is a classic **ambiguity** resolved by matching else to the **nearest if**.
- **Left recursion** `A -> A alpha | beta` breaks **top-down** parsing but is fine for **bottom-up**.
- Eliminating left recursion turns `A -> A a | b` into **A -> b A'** and **A' -> a A' | epsilon**.
- **Left factoring** removes a **common prefix**: `A -> aB1 | aB2` becomes **A -> a A'**, **A' -> B1 | B2**.
- Removing **left recursion** does **not** remove **ambiguity**.
- **Ambiguity** is a property of the **grammar**, not the **language**.

4. Top-Down Parsing - One-Liners

Recursive Descent and Predictive

- **Recursive descent** uses **one procedure per non-terminal** and builds the tree **top-down**.
- General recursive descent may require **backtracking**.
- **Predictive parsing** is recursive descent **without backtracking** using **lookahead**.
- A predictive parser needs the grammar to be **non-left-recursive** and **left-factored**.

LL(1) Parsing

- **LL(1)** means **Left-to-right scan**, **Leftmost derivation**, and **1** symbol lookahead.
- LL(1) parsing is **table-driven** using a **stack** and table **M[A, a]**.
- **FIRST(X)** is the set of terminals that can **begin** a string derived from X.
- **FOLLOW(A)** is the set of terminals that can appear **immediately right** of A.
- **Epsilon** may appear in **FIRST** but **never** in **FOLLOW**.
- The end marker **$** appears in **FOLLOW** (of the start symbol) but **never** in **FIRST**.
- For `A -> alpha`, add the production to **M[A,a]** for each a in **FIRST(alpha)**.
- If **epsilon in FIRST(alpha)**, add `A -> alpha` for each b in **FOLLOW(A)**.
- A grammar is **LL(1)** iff every table cell has at most **one** entry.
- Two LL(1) conditions: **FIRST(alpha) ∩ FIRST(beta) = empty** and, if beta derives epsilon, **FIRST(alpha) ∩ FOLLOW(A) = empty**.
- Every **LL(1)** grammar is **unambiguous**, but not every unambiguous grammar is **LL(1)**.
- A grammar with **left recursion** is **never** LL(1).
- The mnemonic "**FIRST starts, FOLLOW finishes**" recalls their meaning.

Backtracking

- A **backtracking** parser can take **exponential** time and struggles with **side effects**.

- **LL(1)** avoids backtracking by using **1-symbol lookahead**.

5. Bottom-Up Parsing - One-Liners

Handles and Shift-Reduce

- A **handle** is a substring matching a **production RHS** whose reduction is one step of a **rightmost derivation in reverse**.
- **Handle pruning** repeatedly **reduces** handles until the **start symbol** remains.
- Shift-reduce parsing has **4** actions: **shift**, **reduce**, **accept**, and **error**.
- A **shift** pushes the next **input token** onto the **stack**.
- A **reduce** replaces a **handle** on the stack with its **LHS**.
- A **shift/reduce** conflict cannot decide between **shifting** and **reducing**.
- A **reduce/reduce** conflict cannot decide **which production** to reduce by.
- The **dangling-else** causes a **shift/reduce** conflict resolved by **shift**.

LR Parsing Family

- **LR** means **Left-to-right** scan with **Rightmost derivation in reverse**.
- The **4** LR members by power are **LR(0) < SLR(1) < LALR(1) < CLR(1)**.
- An **LR(0) item** is a production with a **dot** like `A -> X . Y Z`.
- LR(0) states are built using **closure** and **GOTO** operations.
- **SLR(1)** reduces `A -> alpha .` only on terminals in **FOLLOW(A)**.
- An **LR(1) item** carries a **lookahead** terminal, written `[A -> a . b, x]`.
- **CLR(1)/canonical LR** is the **most powerful** and has the **largest** table.
- **LALR(1)** merges CLR states with the **same core**, giving **fewer** states.
- LALR has the **same** state count as **SLR** and **LR(0)**.
- State merging in LALR can create **reduce/reduce** conflicts but **never** new **shift/reduce** conflicts.
- **YACC/Bison** generates an **LALR(1)** parser.

Power Comparison

- Power order is **LR(0) < SLR(1) < LALR(1) < CLR(1)**; LL(1) is a **subset** of LR(1).
- Every **LL(1)** grammar is **LR(1)**, but not vice versa.
- If a grammar is **LR(0)** it is automatically **SLR**, **LALR**, and **CLR**.
- The parser with the **most** states is **CLR(1)**.
- The mnemonic for power is **"LR0 < SLR < LALR < CLR"**.

Operator Precedence and YACC

- An **operator grammar** forbids **epsilon** productions and **two adjacent non-terminals**.
- Operator-precedence parsing uses relations **<.**, **.>**, and **=.** between terminals.
- A YACC file has **3** sections separated by %%.
- YACC's default shift/reduce resolution favors **shift**; reduce/reduce favors the **earlier rule**.

6. Syntax-Directed Translation - One-Liners

SDD and Attributes

- An **SDD** attaches **attributes** and **semantic rules** to grammar productions.
- A **synthesized** attribute is computed from a node's **children** (flows **bottom-up**).
- An **inherited** attribute is computed from **parent and siblings** (flows **top-down/sideways**).
- **Terminals** have only **synthesized** attributes.
- The **start symbol** cannot have an **inherited** attribute (no parent).

Attribute Classes and Schemes

- An **S-attributed** SDD uses **only synthesized** attributes and fits **bottom-up/LR** parsing.
- An **L-attributed** SDD allows inherited attributes depending only on **left** symbols and fits **LL** parsing.
- Every **S-attributed** SDD is also **L-attributed**, but not vice versa.
- For S-attributed schemes, semantic actions go at the **end** of the RHS.
- An **annotated parse tree** shows **attribute values** at each node.
- A **dependency graph** dictates evaluation order via **topological sort**.
- A **cyclic** dependency graph means attributes **cannot** be evaluated.

7. Semantic Analysis - One-Liners

Type Checking and Scope

- **Type checking** is the main task of **semantic analysis**.
- **Static** type checking happens at **compile time**; **dynamic** at **run time**.
- **Coercion** is **implicit** conversion; a **cast** is **explicit**.
- **Widening** (small->large) is **safe**; **narrowing** (large->small) risks **data loss**.
- Most languages use **static (lexical)** scope; older Lisp used **dynamic** scope.
- Symbol table operations are **insert**, **lookup**, and **delete**.
- Symbol-table lookup costs: **linear list O(n)**, **BST O(log n)**, **hash O(1)** average.

8. Runtime Environments - One-Liners

Storage and Activation Records

- Runtime memory has **4** areas: **code**, **static**, **heap**, and **stack**.
- The **stack** grows **downward** and the **heap** grows **upward** (toward each other).
- An **activation record** stores **return value**, **parameters**, **control link**, **access link**, **saved status**, and **locals**.
- The **control (dynamic) link** points to the **caller's** frame.
- The **access (static) link** points to the **lexically enclosing** frame.
- **Recursion** requires **stack** allocation, not static allocation.
- Data outliving a procedure goes on the **heap**, managed by **garbage collection** or manual `free`.

Parameter Passing

- There are **4** parameter-passing methods: **call by value**, **reference**, **value-result**, and **name**.
- **Call by value** does **not** affect the caller's variable.
- **Call by reference does** affect the caller and can **swap** two variables.
- **Call by value-result** copies in on **entry** and back on **return**.

- **Call by name** re-evaluates the argument on **each use** (textual substitution).

9. Intermediate Code Generation - One-Liners

IR and Three-Address Code

- The **3** common IRs are **syntax trees**, **DAGs**, and **postfix notation**.
- A **DAG** shares **common subexpressions** that a syntax tree duplicates.
- **Postfix** notation needs **no parentheses** and is evaluated with a **stack**.
- **Three-address code** has at most **one** operator and **three** addresses per statement.
- A copy statement has the form **x = y**; a conditional jump is **if x relop y goto L**.

Representations of TAC

- The **3** TAC representations are **quadruples**, **triples**, and **indirect triples**.
- A **quadruple** has **4** fields: **op, arg1, arg2, result**.
- A **triple** has **3** fields and refers to results by **position/index**.
- **Indirect triples** use a **list of pointers** to triples for easy **reordering**.
- **Triples** are **hard** to reorder; **quadruples** and **indirect triples** are **easy**.

Control Flow, Backpatching, Arrays

- Boolean expressions use **short-circuit** jumping code.
- **Short-circuit**: `A && B` skips B when A is **false**; `A || B` skips B when A is **true**.
- **Backpatching** fills unknown **jump targets** using **truelist**, **falselist**, and **nextlist**.
- The **3** backpatching functions are **makelist**, **merge**, and **backpatch**.
- Backpatching enables **one-pass** code generation despite **forward jumps**.
- A 1-D array element address is **base + index * width**.
- **Row-major** (C) stores rows contiguously; **column-major** (Fortran) stores columns contiguously.
- A function call translates to **param** statements followed by a **call**.

10. Code Optimization - One-Liners

Basic Blocks and Flow Graphs

- A **basic block** has **one entry** and **one exit** with no internal branching.
- A statement is a **leader** if it is the **first** statement, a **jump target**, or **follows a jump**.
- The number of **leaders** equals the number of **basic blocks**.
- A **flow graph** has **basic blocks** as nodes and control-flow **edges**.

Local Optimizations

- **Constant folding** evaluates a **constant expression** at compile time, e.g., `2*3 -> 6`.
- **Constant propagation** substitutes a **variable's known constant** value.
- **Copy propagation** replaces uses of x after `x = y` with **y**.
- **Dead code elimination** removes statements whose results are **never used**.
- **Common subexpression elimination** reuses an already-computed **expression**.
- **Algebraic simplification** applies identities like **x+0=x** and **x*1=x**.

- **Strength reduction** replaces costly ops with cheaper ones, e.g., **x*2 -> x<<1**.
- The most-confused pair is **constant folding** (expression) vs **constant propagation** (variable).

Loop and Peephole Optimizations

- **Code motion** moves **loop-invariant** computations **out of the loop**.
- An **induction variable** changes by a **constant** each iteration.
- **Peephole optimization** works on a **small window** of **target** instructions.
- Peephole techniques include **redundant load/store** removal and **unreachable code** elimination.
- **Loop-invariant code motion** is **machine-independent**.

11. Data Flow Analysis - One-Liners

Reaching Definitions and Liveness

- **Reaching definitions** is a **forward**, **union (may)** analysis.
- A definition **reaches** a point if a path exists with **no redefinition** in between.
- Reaching-definitions equation: **OUT[B] = gen[B] ∪ (IN[B] - kill[B])**.
- **Live variable analysis** is a **backward**, **union (may)** analysis.
- A variable is **live** if its value is **used later** before redefinition.
- Liveness equation: **IN[B] = use[B] ∪ (OUT[B] - def[B])**.
- Liveness is used for **register allocation** and **dead-code elimination**.

Available Expressions and Constant Propagation

- **Available expressions** is a **forward**, **intersection (must)** analysis.
- An expression is **available** if **every** path computes it with no operand redefinition.
- Available expressions is used for **global CSE**.
- **Constant propagation** analysis uses a **lattice** with values **UNDEF, constant, NAC**.
- The only **intersection (must)** data-flow analysis among the four is **available expressions**.
- The most-confused data-flow pair is **live (backward/union)** vs **available (forward/intersection)**.
- Mnemonic: **Reaching and Available go forward; Live goes backward**.

12. Code Generation - One-Liners

Issues and Register Allocation

- The **3** code-generation issues are **instruction selection**, **register allocation**, and **instruction ordering**.
- **Register allocation** decides which values stay in **registers**; **assignment** picks the **specific register**.
- Register allocation via **graph coloring** is **NP-complete**.
- A **k-colorable** interference graph fits in **k** registers.
- **Spilling** stores a value to **memory** when registers run out.
- A simple code generator tracks **register descriptors** and **address descriptors**.
- The **Sethi-Ullman** labeling computes the **minimum registers** for an expression tree.
- A **DAG** for a basic block exposes **common subexpressions** for better code.

13. Exam Focus - One-Liners

Rapid Recall

- An **invalid token** is a **lexical** error; a **missing semicolon** is a **syntax** error.
- A **type mismatch** and an **undeclared variable** are **semantic** errors.
- An **LL(1)** conflict is a table cell with **2** entries.
- An **LR** conflict is a **shift/reduce** or **reduce/reduce** clash.
- The parser power order is **LR(0) < SLR(1) < LALR(1) < CLR(1)**.
- **YACC** produces an **LALR(1)** parser.
- **S-attributed** is a **subset** of **L-attributed**.
- A **quadruple** has **4** fields and a **triple** has **3**.
- The number of **basic blocks** equals the number of **leaders**.
- **Live variables** is **backward/union**; **available expressions** is **forward/intersection**.

14. Compiler Overview - Extended One-Liners

- The translator with the **highest** execution speed is the **compiler** because translation is paid **once**.
- The **symbol table** is **not** counted among the **6** phases of a compiler.
- A **macro** is expanded by the **preprocessor**, not the compiler proper.
- The output of the **assembler** is **object code** (machine code).
- A **T-diagram** describes a translator's **source**, **target**, and **implementation** languages.
- A **native compiler** has **host == target**; a **cross-compiler** has **host != target**.
- The 3 main reasons for **multi-pass** compilers are **forward references**, **better optimization**, and **limited memory** machines.
- The component that performs **error recovery** is the **error handler**.
- A **disassembler** converts **machine code** back to **assembly**.
- The **IR** sits between the **front end** and the **back end**.
- The earliest phase to detect an **illegal character** is **lexical analysis**.
- The phase that uses a **CFG** as its specification is **syntax analysis**.
- The number of distinct error types by phase tested in CIL MT is **3**: **lexical**, **syntactic**, **semantic**.

15. Lexical Analysis - Extended One-Liners

- The smallest unit returned by the scanner is a **token**.
- **Comments** and **whitespace** produce **no** tokens.
- The number of LEX sections is **3**, delimited by %%.
- A **relop** token covers operators like **<, <=, =, !=, >, >=**.
- An **unterminated string** is a **lexical** error.
- The pattern for an integer constant is **digit+**.
- The transformation that reduces DFA size is **DFA minimization**.
- **Maximal munch** means the scanner takes the **longest** matching lexeme.
- The two scanner pointers are **lexemeBegin** and **forward**.
- A finite automaton that recognizes tokens has a **finite** number of **states**.
- The class of languages tokens belong to is **regular**.
- **Subset construction** can produce up to **2^n** DFA states from an n-state NFA.
- The scanner returns a **symbol-table pointer** as the attribute for an **identifier**.
- **Flex** is the GNU implementation of **LEX**.

16. Parsing Grammars - Extended One-Liners

- A **sentential form** is any string derivable from the **start symbol**.
- A **sentence** is a sentential form consisting only of **terminals**.
- The set of all sentences is the **language** L(G) generated by grammar G.
- A grammar is **recursive** if a non-terminal can derive a string containing **itself**.
- **Right recursion** is acceptable for **top-down** parsers.
- A **useless symbol** is either **non-generating** or **non-reachable**.
- **Epsilon productions** have the form **A -> epsilon**.
- **Unit productions** have the form **A -> B** (one non-terminal).
- **Chomsky hierarchy** places CFGs at **Type-2** and regular grammars at **Type-3**.
- Two grammars are **equivalent** if they generate the **same language**.

17. Top-Down Parsing - Extended One-Liners

- A predictive parser uses **lookahead** to pick a **unique** production.
- The stack of an LL(1) parser initially holds **$** and the **start symbol**.
- An LL(1) parse **accepts** when both stack and input show **$**.
- A blank entry in the LL(1) table denotes an **error**.
- The 2 grammar transformations needed before LL(1) are **left-recursion removal** and **left factoring**.
- **FIRST** of a terminal a is **{a}** .
- **FIRST** of epsilon is **{epsilon}** .
- **FOLLOW** of the start symbol always includes **$**.
- If a non-terminal derives **epsilon**, FIRST contains **epsilon**.
- An LL(1) table cell with **2** productions indicates a **non-LL(1)** grammar.
- The number of lookahead symbols in LL(1) is **1**.
- A **recursive-descent** parser without backtracking is a **predictive** parser.

18. Bottom-Up Parsing - Extended One-Liners

- The **viable prefixes** of a grammar are recognized by the **LR(0) automaton**.
- A **kernel item** is one whose dot is **not** at the **left end** (plus the augmented start item).
- The **augmented grammar** adds a new start production **S' -> S**.
- **Acceptance** in LR occurs on reducing by **S' -> S** with lookahead **$**.
- The **GOTO** table handles **non-terminal** transitions; the **ACTION** table handles **terminals**.
- An **LR(0) automaton** state with a complete item triggers a **reduce**.
- **SLR** resolves some LR(0) conflicts using **FOLLOW** sets.
- The number of LR parser tables is **2**: **ACTION** and **GOTO**.
- **CLR** distinguishes states that **SLR** and **LALR** merge, avoiding conflicts.
- For most practical grammars, **LALR(1)** suffices, which is why **YACC** uses it.
- A grammar can be **LALR** but not **SLR**, and **CLR** but not **LALR**.
- Bottom-up parsers handle **left recursion** naturally.
- The most **memory-efficient** powerful parser is **LALR(1)**.
- The 4 LR table entry types are **shift**, **reduce**, **accept**, and **blank/error**.

19. SDT - Extended One-Liners

- **Semantic rules** compute **attribute values** from other attributes.
- An attribute computed only from **constants and children** is **synthesized**.
- **L-attributed** SDDs permit a single **left-to-right depth-first** pass.
- **S-attributed** SDDs are evaluated during **bottom-up** parsing.
- A **postfix** SDT places all actions at the **ends** of production bodies.
- **Inherited** attributes are convenient for passing **type** information **down** a declaration list.
- The evaluation order for an SDD is any **topological sort** of its **dependency graph**.
- An SDD with a **cyclic** dependency graph is **not** evaluable.
- **Markers/nonterminals** are sometimes inserted to make a scheme **L-attributed**.

20. Semantic & Runtime - Extended One-Liners

- A **type expression** can be **basic** or **constructed** (array, record, pointer, function).
- A **strongly typed** language catches all **type errors** before runtime.
- **Name equivalence** vs **structural equivalence** are 2 ways to compare types.
- The **environment** maps a name to a **storage location**; the **state** maps a location to a **value**.
- A **binding** associates a **name** with a **value/location**.
- **Static allocation** fixes addresses at **compile time** and forbids **recursion**.
- The **display** is an array of pointers speeding **non-local** access.
- **Garbage collection** reclaims **heap** memory with no references.
- **Dangling pointer** and **memory leak** are 2 classic **heap** bugs.
- **Call by reference** passes an **address**; **call by value** passes a **copy**.
- **Activation tree** depicts the **nesting** of procedure activations.
- The **return address** is part of the **saved machine status** in a frame.

21. Intermediate Code - Extended One-Liners

- A **syntax tree** condenses a **parse tree** by removing single-production chains.
- A **DAG** for a+a has the leaf a referenced **twice** with **one** node.
- The 3 address parts of TAC are **result**, **operand1**, and **operand2**.
- A **label** marks a **jump target** in TAC.
- The TAC for `a = b + c * d` uses **2** temporaries: **t1** and **t2**.
- A **triple** avoids explicit **temporary names**.
- **Indirect triples** add a level of **indirection** via a **statement list**.
- **Postfix** (Reverse Polish) for a+b*c is **a b c * +**.
- For a 2-D array, **row-major** address uses **base + (i*n2 + j)*w**.
- Function arguments are emitted with the **param** instruction.
- The 3 boolean-expression code styles are **numerical**, **short-circuit**, and **control-flow**.
- **Backpatching** requires **one** extra pass to be **avoided**, achieving **single-pass** translation.

22. Optimization - Extended One-Liners

- The **first** statement of a procedure is always a **leader**.
- Each **jump target** starts a **new basic block**.

- **Local** optimization stays within a **basic block**; **global** spans the **flow graph**.
- **Loop unrolling** reduces **loop control** overhead.
- **Loop fusion** merges **adjacent** loops with the same bounds.
- **Induction-variable elimination** removes redundant **loop counters**.
- **x = x + 0** is removed by **algebraic simplification**.
- **Dead code** after a `return` is **unreachable code**.
- **CSE** typically pairs with **copy propagation** and **dead-code elimination**.
- **Machine-independent** optimization works on **IR**; **machine-dependent** works on **target code**.
- The optimization moving `i*4` outside a loop is **loop-invariant code motion**.
- **Peephole** optimization operates over a **sliding window** of a **few** instructions.

23. Data Flow - Extended One-Liners

- A **data-flow framework** has a **direction**, a **transfer function**, and a **meet operator**.
- **Forward** analyses compute **OUT** from **IN**; **backward** analyses compute **IN** from **OUT**.
- The **meet** for **may** analyses is **union**; for **must** analyses it is **intersection**.
- **Reaching definitions** = **forward** + **union**.
- **Available expressions** = **forward** + **intersection**.
- **Live variables** = **backward** + **union**.
- **Very busy expressions** = **backward** + **intersection**.
- A **gen** set lists facts **created** in a block; **kill** lists facts **destroyed**.
- Data-flow equations are solved by **iterative** fixpoint computation.
- **Reaching definitions** detects **uninitialized variable** use.
- **Liveness** drives **register allocation** via the **interference graph**.
- The 4 classic data-flow problems split 2 **forward** and 2 **backward**.

24. Code Generation - Extended One-Liners

- The **getReg** function selects a **register** for an operation.
- A **register descriptor** tracks **what each register holds**.
- An **address descriptor** tracks **where a variable's value resides**.
- **Spill** code adds **store** and **load** instructions.
- **Sethi-Ullman** numbering minimizes **registers** with **no spills**.
- **Instruction selection** maps **IR** to **target instructions**.
- **Instruction scheduling** reorders to improve **pipeline** usage.
- A **2-address** or **3-address** target machine affects **code shape**.
- The interference graph being **k-colorable** means **k** registers suffice.
- **Graph-coloring** register allocation is **NP-complete**.

25. Final Rapid-Fire - Extended One-Liners

- The phase order mnemonic is **"La Sa Se In Op Co"**.
- The parser power mnemonic is **"LR0 < SLR < LALR < CLR"**.
- **Epsilon** is in **FIRST** only; **$** is in **FOLLOW** only.
- A **handle** relates to **bottom-up** parsing.
- A **basic block** count equals the **leader** count.

- **YACC** = **LALR(1)**; **LEX** = **regex-based scanner**.
- **Quadruple** = **4** fields; **triple** = **3**.
- **S-attributed** ⊆ **L-attributed**.
- **Live** = **backward**; **available** = **forward**.
- **Coercion** = **implicit**; **cast** = **explicit**.
- **Stack** grows **down**; **heap** grows **up**.
- **Recursion** needs the **stack**, not **static** allocation.
- **CLR** has the **most** states; **SLR=LALR=LR0** share state counts.
- **Backpatching** enables **one-pass** flow-of-control translation.
- **Strength reduction**: **x*2 -> x<<1**.
- **DAG** reveals **common subexpressions**.
- **Type checking** is done in **semantic analysis**.
- **Constant folding** works on **expressions**; **constant propagation** on **variables**.
- **Panic-mode** recovery uses **synchronizing tokens**.
- The compiler front end ends at **intermediate code generation**.

26. Phases Deep Dive - More One-Liners

- The lexical analyzer is also called a **scanner** or **tokenizer**.
- The syntax analyzer is also called a **parser**.
- The output of **lexical analysis** feeds the **parser**.
- The **semantic analyzer** consults the **symbol table** for **type** info.
- **Intermediate code** is both **machine-independent** and easy to **optimize**.
- The **optimizer** must **preserve** the program's **meaning**.
- **Code generation** targets a specific **instruction set architecture**.
- A **logical** or run-time error like **divide-by-zero** is generally **not** caught at compile time.
- The 6-phase model is from the **Dragon Book** (Aho, Lam, Sethi, Ullman).
- **Lexical + syntax + semantic** together form the **analysis** part.
- **Intermediate** + **optimization** + **code gen** lean toward **synthesis**.
- A **single-pass** C-like compiler requires **declaration before use**.

27. Automata & Regex - More One-Liners

- A **DFA** has **exactly one** transition per (state, symbol) pair.
- An **NFA** may have **zero or many** transitions and **epsilon** moves.
- Every **NFA** has an equivalent **DFA** (Rabin-Scott theorem).
- A **regular language** is closed under **union**, **concatenation**, and **Kleene star**.
- **Identifiers**, **numbers**, and **keywords** are all **regular**.
- The empty string is denoted **epsilon** and the empty set **phi**.
- A **minimized DFA** is **unique** up to **state renaming**.
- A scanner generator compiles regex into a **transition table**.
- **Longest-match** plus **rule-priority** resolves scanner ambiguity.

28. LL Parsing - More One-Liners

- The LL(1) driver pops a **matched terminal** and pushes a production's **RHS reversed**.

- A **terminal** on top matching the input causes a **match (pop)**.
- A **non-terminal** on top triggers a **table lookup**.
- An **empty** table cell signals a **syntax error**.
- LL(1) cannot parse **left-recursive** grammars.
- LL(1) cannot parse most **ambiguous** grammars.
- LL(k) uses **k** lookahead symbols; CIL focuses on **k=1**.
- **Strong LL(k)** bases decisions purely on **lookahead**.
- The number of LL(1) table dimensions is **2**: **non-terminals** x **terminals+$**.

29. LR Parsing - More One-Liners

- **Closure** adds items for non-terminals right after the **dot**.
- **GOTO(I, X)** moves the **dot** past symbol **X**.
- The starting state is **closure** of **S' -> . S**.
- A **shift** action moves to a new **state** and pushes it.
- A **reduce** action pops **|RHS|** states/symbols.
- LR parsers store **states** on the stack, not just symbols.
- An **inadequate state** contains a **conflict**.
- **SLR** uses **FOLLOW**; **CLR/LALR** use **precise lookahead**.
- LALR merges **cores**; this can never add a **shift/reduce** conflict.
- The most **compact** adequate parser table is usually **LALR(1)**.
- **Bison** is the GNU version of **YACC**.
- An **LR(1)** item set count is **>=** the **LALR(1)** count.

30. Translation & IR - More One-Liners

- **E.place** holds the name with an expression's **value**.
- **E.code** holds the **generated** TAC for an expression.
- A new temporary is created with **newtemp()**.
- A new label is created with **newlabel()**.
- **gen()** emits a **three-address** statement.
- A **while** loop needs **2** labels: **begin** and **after**.
- An **if-then** needs **1** jump on the **false** condition.
- **Short-circuit** && evaluates the right operand only if left is **true**.
- A **call** statement may follow **n param** statements for n arguments.
- **Triples** reference temporaries by their **line number**.

31. Optimization & Data Flow - More One-Liners

- A **def** of a variable **kills** all prior **reaching definitions** of it.
- **gen** for liveness is the **use** set; **kill** is the **def** set.
- **Available expressions** kills an expression when an **operand** is redefined.
- **Iterative** data-flow converges at a **fixpoint**.
- **Forward** problems initialize **OUT**; **backward** problems initialize **IN**.
- **Reaching definitions** is used by **constant propagation**.
- A **flow graph** has a unique **entry** and **exit** node by convention.

- A **loop** in a flow graph has a **back edge** to a **header**.
- **Dominators** help identify **natural loops**.
- **Copy propagation** is enabled by **reaching definitions** of copies.

32. Final Definitions Burst - More One-Liners

- A **token** is a **terminal symbol** with a **lexical category**.
- A **handle** is the **RHS** reduced in a **rightmost-in-reverse** derivation.
- A **basic block** is a straight-line code sequence with **single** entry/exit.
- A **leader** starts a **basic block**.
- A **DAG** is a **directed acyclic graph** sharing **subexpressions**.
- An **activation record** holds the state of **one** procedure call.
- **Backpatching** resolves **forward jump** targets later.
- **Synthesized** attributes flow **up**; **inherited** flow **down**.
- **Three-address code** has **<=1** operator per instruction.
- A **parse tree** root is the **start symbol**.
- The **LR** family parses **bottom-up**; the **LL** family parses **top-down**.
- **Spilling** occurs when **registers** are insufficient.
- A **cross-compiler** runs on **host** and targets a **different** machine.
- **Coercion** is performed by the **compiler** automatically.
- The **symbol table** interacts with **all 6** phases.
- **Peephole** optimization is usually **machine-dependent**.
- **Sethi-Ullman** minimizes **register** usage for **expression trees**.
- **Available expressions** powers **global common-subexpression elimination**.
- **LALR(1)** is the parser used by **YACC/Bison**.
- **FIRST** and **FOLLOW** build the **LL(1)** and **SLR** tables.
- A **useless symbol** is removed during grammar **simplification**.
- **Ambiguity** is detected as **multiple** parse trees, not via an algorithm in general.
- **Strength reduction** converts **multiplication** to **addition/shift**.
- **Constant folding** happens at **compile time**.
- **Dead-code elimination** relies on **liveness** analysis.
- The **interference graph** models **simultaneously-live** variables.
- **Row-major** is used by **C**; **column-major** by **Fortran**.
- A **quadruple** explicitly names its **result**; a **triple** does **not**.
- **Indirect triples** ease **instruction reordering** during optimization.
- The number of **LR** actions is **4**: **shift, reduce, accept, error**.

33. Numeric & Comparison Burst - More One-Liners

- The number of compiler phases is **6**; the number of error types tested is **3**.
- A quadruple has **4** fields; a triple has **3**; indirect triples add a **pointer list**.
- LL uses **1** lookahead; LR(0) uses **0**; SLR/LALR/CLR use **1**.
- There are **4** LR parser variants and **4** shift-reduce actions.
- The 2 LR tables are **ACTION** and **GOTO**.
- The 4 data-flow problems are **reaching definitions**, **available expressions**, **live variables**, and **very busy expressions**.

- Among data-flow problems, **2** are forward and **2** are backward.
- There are **4** parameter-passing methods and **4** memory areas at runtime.
- A LEX program and a YACC program both have **3** sections.
- Hash-table symbol lookup is **O(1)** average vs **O(log n)** for a BST.
- The 3 IR forms are **trees**, **DAGs**, and **postfix**.
- The 6 local optimizations include **folding**, **propagation**, **copy propagation**, **dead code**, **CSE**, and **strength reduction**.
- The 3 leader rules are **first statement**, **jump target**, and **post-jump statement**.
- **Power**: LL(1) ⊂ LR(1); **LR(0) < SLR < LALR < CLR**.
- A DFA from an NFA may need up to **2^n** states.

34. Tricky Distinctions - More One-Liners

- **Lexeme** is concrete text; **token** is its category; **pattern** is the matching rule.
- **Synthesized** comes from children; **inherited** comes from parent/siblings.
- **S-attributed** fits **bottom-up**; **L-attributed** fits **top-down**.
- **Control link** is dynamic (caller); **access link** is static (enclosing scope).
- **Call by value** can't swap; **call by reference** can.
- **Constant folding** acts on a constant **expression**; **constant propagation** on a **variable**.
- **Live variables** is backward/union; **available expressions** is forward/intersection.
- **Quadruples** reorder easily; **triples** do not.
- **LALR** can add **reduce/reduce** conflicts; never new **shift/reduce**.
- **Left recursion** hurts **top-down**; helps **bottom-up**.
- **Epsilon** is in FIRST only; **$** is in FOLLOW only.
- **Coercion** is implicit; **casting** is explicit.
- **Widening** is safe; **narrowing** risks loss.
- **Static** scope uses program text; **dynamic** scope uses the call chain.
- **Machine-independent** optimization is on IR; **peephole** is on target code.

35. CIL MT Direct-Answer Burst - More One-Liners

- The phase that builds the **parse tree** is **syntax analysis**.
- The phase that does **type checking** is **semantic analysis**.
- The tool generating an LALR(1) parser is **YACC**.
- The tool generating a scanner is **LEX**.
- The data structure storing identifier attributes is the **symbol table**.
- The substring reduced in bottom-up parsing is the **handle**.
- The graph used for register allocation is the **interference graph**.
- The algorithm minimizing registers for expression trees is **Sethi-Ullman**.
- The IR sharing common subexpressions is the **DAG**.
- The technique resolving forward jumps is **backpatching**.
- The conflict in dangling-else is **shift/reduce**, resolved by **shift**.
- The set used by SLR for reductions is **FOLLOW**.
- The attribute type terminals carry is **synthesized**.
- The memory area for dynamic allocation is the **heap**.
- The memory area for activation records is the **stack**.
- The number of addresses in TAC is at most **3**.

- The optimization replacing `2*3` with `6` is **constant folding**.
- The analysis detecting uninitialized use is **reaching definitions**.
- The recovery method using synchronizing tokens is **panic mode**.
- The translator running statement-by-statement is the **interpreter**.